

# IKT215 Assignment 3: Function Approximation Classification, Error Estimation and Model Selection

**Student:** Gomery K. Wanjiru  
**Course:** IKT215

|                   |           |                   |           |
|-------------------|-----------|-------------------|-----------|
| <b>Disclaimer</b> | <b>on</b> | <b>Generative</b> | <b>AI</b> |
|-------------------|-----------|-------------------|-----------|

---

In accordance with UiA's guidelines, I declare that I used Generative AI tools to assist in structuring the LaTeX report and generating boilerplate code for plotting. The neural network implementation, error estimation analysis, model selection methodology, and interpretation of results are my own work, guided by the course lecture materials (Lectures 6–8) and exercises (7–9) + class attendance.

## Introduction

---

This report presents a neural network-based classification system for the mfeat-factors dataset, which contains 216 profile correlation features extracted from handwritten digits (0–9) from Dutch utility maps. The assignment covers four parts: data exploration and preparation, neural network implementation, comparison of error estimation techniques, and model selection strategies.

All experiments use PyTorch for neural network implementation and scikit-learn for data preprocessing and evaluation utilities. The dataset is pre-split into 1600 training and 400 test samples, with perfectly balanced classes (160/40 samples per class for train/test respectively).

## Part 1: Setup and Exploration

---

The mfeat-factors dataset was loaded from the provided `.npz` files. Table 1 summarizes the dataset characteristics.

Table 1: Dataset summary.

| Property          | Training Set | Test Set |
|-------------------|--------------|----------|
| Samples           | 1600         | 400      |
| Features          | 216          | 216      |
| Classes           | 10 (0–9)     | 10 (0–9) |
| Samples per class | 160          | 40       |
| Feature range     | [0, 1353]    | —        |

**Normalization.** Since the raw features span a wide range ( $[0, 1353]$ , mean  $\approx 318$ , std  $\approx 352$ ), I applied **standardization** (zero mean, unit variance) using `StandardScaler`. The scaler was fitted on the training data only to prevent information leakage. After normalization, the training set has mean  $\approx 0.0$  and std  $\approx 1.0$ , while the test set has mean  $\approx -0.005$  and std  $\approx 1.007$ , confirming proper scaling. The classes are perfectly balanced, so no resampling was needed.

## Part 2: Neural Network Implementation

---

### Architecture

I implemented a fully connected neural network in PyTorch with the following architecture:

Table 2: Network architecture (primary model, Config B).

| Layer           | Input | Output | Details                       |
|-----------------|-------|--------|-------------------------------|
| Linear + BN     | 216   | 128    | BatchNorm, ReLU, Dropout(0.3) |
| Linear + BN     | 128   | 64     | BatchNorm, ReLU, Dropout(0.3) |
| Linear (output) | 64    | 10     | Raw logits (softmax in loss)  |

Total trainable parameters: **37,066**. The design choices are motivated by the lectures:

- **ReLU activation:** Non-saturating nonlinearity that avoids the vanishing gradient problem (Lecture 6, slide 50).
- **Batch normalization:** Stabilizes training by normalizing layer inputs, enabling higher learning rates.
- **Dropout (0.3):** Regularization to reduce overfitting, acting as an implicit ensemble.
- **Cross-entropy loss:** Appropriate for multi-class classification using softmax outputs (Lecture 6, slide 41–42).

## Training Approach

- **Optimizer:** Adam with learning rate  $\eta = 0.001$  and weight decay  $10^{-4}$ .
- **Batch size:** 64 (mini-batch SGD for faster convergence, Lecture 6, slide 52).
- **Learning rate schedule:** ReduceLROnPlateau (halves LR after 10 epochs without improvement).
- **Early stopping:** Training stops after 25 epochs without validation loss improvement, and the best model (by validation loss) is restored.
- **Validation split:** 80/20 stratified split of training data.

The model converged within  $\sim 105$  epochs. Training and validation curves (Figure 1) show smooth convergence with no significant overfitting gap.

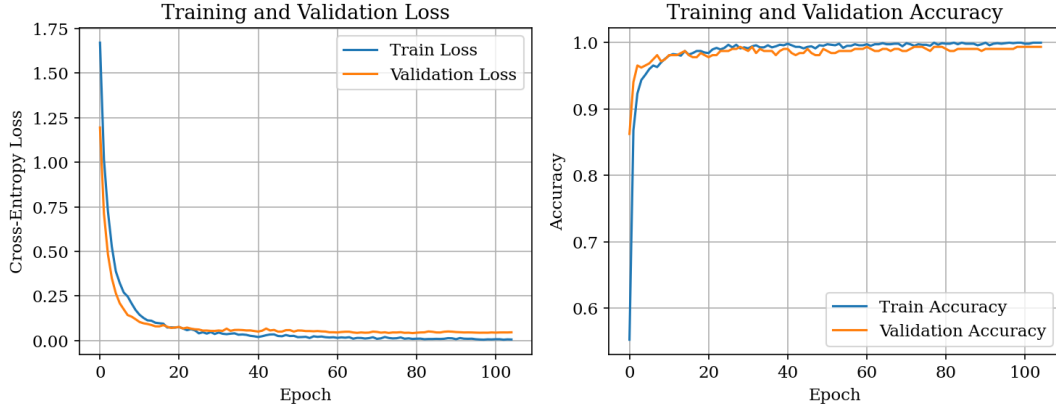


Figure 1: Training and validation loss/accuracy curves. The model converges around epoch 50 with early stopping triggered at epoch 105.

**Test performance:** The model achieves **97.75%** accuracy on the held-out test set (error rate: 2.25%). Figure 2 shows the confusion matrix. The most error-prone classes are digit 1 (3 errors: confused with 3 and 7) and digit 5 (3 errors: confused with 3, 4, and 9), which share stroke-level similarities in handwritten form.

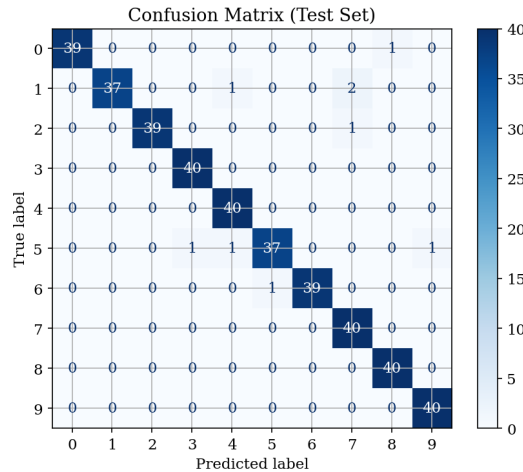


Figure 2: Confusion matrix on the test set. Most classes achieve near-perfect classification.

## Part 3: Error Estimation

I implemented three error estimation techniques using the same network architecture (Config B: 128-64, dropout 0.3) and compared them against the true test error.

## Resubstitution Error

The classifier is trained on *all*  $n = 1600$  training samples and evaluated on the *same* training data (Lecture 7, slide 7):

$$\Xi_n(\Psi_n, S_n) = \frac{1}{n} \sum_{i=1}^n |Y_i - \Psi_n(S_n)(\mathbf{X}_i)|$$

**Result:** Resubstitution error = **0.00%** (accuracy 100%).

The model achieves perfect training accuracy, a dramatically optimistic estimate compared to the true test error of 1.75%. As discussed in Lecture 7 (slide 23), the resubstitution estimator has  $\text{Bias}(\hat{\varepsilon}_n) < 0$  (optimistically biased) because the classifier is evaluated on the same data used for design. A neural network with 37,066 parameters and only 1,600 samples has ample capacity to memorize the training set, producing  $\hat{\varepsilon}_n^{\text{resub}} = 0$  even when the true error is substantially higher.

**Advantage:** Fastest estimation; no additional data needed. **Limitation:** Severely underestimates the true error, especially for high-capacity models; unacceptably large bias in small-sample situations (Lecture 7, slide 23).

## Hold-Out Error

The training data is split 80/20 into a design set ( $n-m = 1280$  samples) and a hold-out set ( $m = 320$  samples). The classifier is trained on the design set and evaluated on the hold-out set (Lecture 7, slide 18–21).

**Result:** Hold-out error = **0.62%** (accuracy 99.38%).

While the hold-out estimator is unbiased given the training data,  $E[\hat{\varepsilon}_{n,m} | S_n] = \varepsilon_n$  (Lecture 7, slide 19), its RMS is distribution-independently bounded by (Lecture 7, slide 20):

$$\text{RMS}(\hat{\varepsilon}_{n,m}) \leq \frac{1}{2\sqrt{m}} = \frac{1}{2\sqrt{320}} \approx 2.80\%$$

This means the hold-out estimate could deviate from the true error by up to 2.80% in the worst case. The observed deviation is  $|0.62\% - 1.75\%| = 1.13\%$ , within this bound.

The hold-out error (0.62%) is noticeably lower than the true test error (1.75%). One likely reason is that the hold-out set comes from the same pool of 2000 samples as the training data — so the split is random within one collection, whereas the external test set was provided separately and may capture slightly different variation. On top of that, we only train on 1280 samples here instead of the full 1600, but since the hold-out portion “looks like” the training portion, the model ends up appearing better on it than it really is on unseen data.

**Advantage:** Unbiased conditional on training data; bounded RMS. **Limitation:** Reduces training data; high variance from a single split; estimate depends on the particular partition chosen.

## 5-Fold Cross-Validation

The training data is partitioned into 5 equal folds. For each fold, the model is trained on 4 folds (1280 samples) and evaluated on the remaining fold (320 samples). The average error across folds gives the CV estimate (Lecture 7, slide 9).

**Result:** CV error = **2.50% ± 0.44%**.

Table 3 shows the per-fold breakdown, demonstrating moderate stability across partitions.

| Table 3: Per-fold 5-fold cross-validation results. |            |           |                                     |
|----------------------------------------------------|------------|-----------|-------------------------------------|
| Fold                                               | Train Size | Test Size | Error Rate                          |
| 1                                                  | 1280       | 320       | 2.19%                               |
| 2                                                  | 1280       | 320       | 2.81%                               |
| 3                                                  | 1280       | 320       | 1.88%                               |
| 4                                                  | 1280       | 320       | 3.12%                               |
| 5                                                  | 1280       | 320       | 2.50%                               |
| <b>Mean <math>\pm</math> Std</b>                   |            |           | <b>2.50% <math>\pm</math> 0.44%</b> |

This is the closest estimate to the true test error (1.75%). As expected from theory (Lecture 7, slides 24–25),  $k$ -fold CV has near-unbiasedness:  $E[\hat{\epsilon}_n] = E[\epsilon_{n-n/k}]$ . Since each fold trains on only 80% of the data ( $n - n/k = 1280$ ), the CV estimate is *pessimistically* biased:  $E[\epsilon_{1280}] > E[\epsilon_{1600}]$ , because a classifier trained on fewer samples generally has higher error. This explains why the CV error (2.50%) slightly exceeds the true test error (1.75%). The fold-to-fold standard deviation of 0.44% reflects the variance inherent in both the random partitioning and the instability of neural network training.

**Advantage:** Uses all data for both training and evaluation; near-unbiased; provides variance estimate. **Limitation:** Computationally expensive ( $k \times$  training); pessimistically biased due to smaller training sets; variance can be large for unstable classifiers (Lecture 7, slide 26).

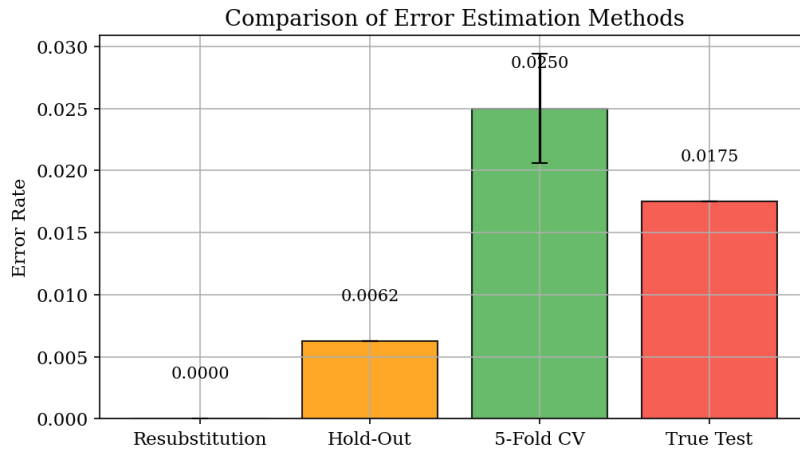


Figure 3: Comparison of error estimation methods. Resubstitution achieves 0% error (severe optimistic bias), hold-out underestimates at 0.62%, and 5-fold CV (2.50%) closely brackets the true test error (1.75%) with slight pessimistic bias.

## Summary

| Table 4: Error estimation results. |                   |                      |
|------------------------------------|-------------------|----------------------|
| Method                             | Error Rate        | Bias Direction       |
| Resubstitution                     | 0.00%             | Severely Optimistic  |
| Hold-Out (80/20)                   | 0.62%             | Optimistic           |
| 5-Fold CV                          | 2.50% $\pm$ 0.44% | Slightly pessimistic |
| True Test Error                    | 1.75%             | —                    |

The results confirm the theoretical hierarchy: resubstitution is severely optimistically biased (Bias  $< 0$ , Lecture 7, slide 23), hold-out improves on this but remains optimistic due to single-split variance,

and cross-validation provides the most reliable estimate by averaging over multiple splits. The RMS bound for hold-out with  $m = 320$  is  $\leq 2.80\%$  (Lecture 7, slide 20), and the CV pessimistic bias  $E[\varepsilon_{1280}] > E[\varepsilon_{1600}]$  explains why CV slightly overshoots the true error.

## Part 4: Model Selection

I compared three network configurations (Table 5) using two model selection approaches.

Table 5: Network configurations compared for model selection.

| Configuration     | Hidden 1 | Hidden 2 | Dropout | LR     | Parameters |
|-------------------|----------|----------|---------|--------|------------|
| Config A (Small)  | 64       | 32       | 0.2     | 0.001  | 16,490     |
| Config B (Medium) | 128      | 64       | 0.3     | 0.001  | 37,066     |
| Config C (Large)  | 256      | 128      | 0.4     | 0.0005 | 90,506     |

### Train-Validation-Test Approach

The training data was split into three sets: training (70%, 1120 samples), validation (15%, 240 samples), and an internal test set (15%, 240 samples). Each configuration was trained on the training portion, the best model was selected by validation accuracy, and the final performance was assessed on the external test set (Lecture 8, slide 16).

Table 6: Train-Validation-Test results.

| Configuration     | Val Acc       | Internal Test Acc | Final Test Acc |
|-------------------|---------------|-------------------|----------------|
| Config A (Small)  | 0.9917        | 0.9833            | <b>0.9800</b>  |
| Config B (Medium) | <b>1.0000</b> | 0.9792            | 0.9750         |
| Config C (Large)  | 0.9917        | 0.9833            | 0.9700         |

Config B was selected with the highest validation accuracy (100%). The final test accuracy is 97.50%. However, by the selection probability theorem (Lecture 8, slide 14), the probability of making a wrong selection satisfies:

$$P(\varepsilon(\psi_n) - \varepsilon(\psi_n^*) > \tau) \leq 2Ne^{-m\tau^2/2}$$

Plugging in  $N = 3$  classifiers,  $m = 240$  validation samples, and  $\tau = 0.01$  (a 1% error difference), we get  $P \leq 2 \cdot 3 \cdot e^{-240 \cdot 0.0001/2} = 6 \cdot e^{-0.012} \approx 5.93$ . Since this exceeds 1, the bound tells us nothing useful — 240 samples simply are not enough to reliably tell apart models whose true errors differ by less than 1%. This lines up with what we observed: the TVT approach could barely distinguish the three configurations.

### Cross-Validation Approach

Each configuration was evaluated using 5-fold cross-validation on the full training set, then the best configuration was retrained on all training data and evaluated on the test set (Lecture 8, slide 18).

Table 7: Cross-Validation model selection results with per-fold accuracy.

| Configuration | F1    | F2    | F3    | F4    | F5    | Mean $\pm$ Std                      |
|---------------|-------|-------|-------|-------|-------|-------------------------------------|
| Config A      | .9781 | .9812 | .9844 | .9781 | .9625 | .9769 $\pm$ .0076                   |
| Config B      | .9781 | .9812 | .9875 | .9781 | .9719 | .9794 $\pm$ .0051                   |
| Config C      | .9781 | .9812 | .9844 | .9812 | .9781 | <b>.9806 <math>\pm</math> .0023</b> |

Config C (Large) was selected with the highest mean CV accuracy (98.06%) and, notably, the *lowest* variance ( $\sigma = 0.23\%$ ). After retraining on the full training set, it achieves **97.75%** on the test set. The per-fold breakdown reveals that all three configurations perform similarly on Folds 1–3, but Config C is more stable on the harder Folds 4–5, where Configs A and B drop more sharply. Config C benefits from higher capacity (90,506 parameters) paired with stronger regularization (dropout 0.4), which provides robustness across different data partitions.

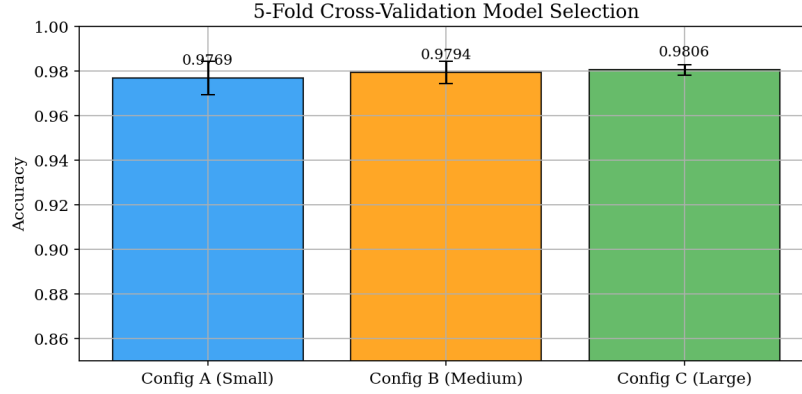


Figure 4: 5-Fold CV accuracy for the three configurations. Config C achieves the highest mean accuracy with the lowest variance.

## Comparison of Selection Approaches

The TVT approach selected Config B (97.50% test accuracy) while CV selected Config C (97.75%), demonstrating that CV provided a more discriminating selection. The TVT approach suffered from a validation set too small to distinguish models whose true errors differ by less than 1%. CV uses all data for evaluation, providing more reliable estimates aligned with the validation error minimization theory (Lecture 8, slide 11–15).

This result also illustrates the **complexity–sample size tradeoff** (Lecture 8, slides 7–8). The error of a designed classifier decomposes as  $\varepsilon_{n,\mathcal{C}} = \varepsilon^* + \Delta_{\mathcal{C}} + \Delta_{n,\mathcal{C}}$ , where  $\Delta_{\mathcal{C}}$  is the approximation error (smaller for more complex  $\mathcal{C}$ ) and  $\Delta_{n,\mathcal{C}}$  is the design error (larger for more complex  $\mathcal{C}$  at fixed  $n$ ). With  $n = 1600$  samples, Config C’s bigger model (90K parameters) brings  $\Delta_{\mathcal{C}}$  down without blowing up  $\Delta_{n,\mathcal{C}}$ , since the strong dropout (0.4) keeps the effective complexity in check. With a smaller dataset, the simpler Config A would probably win out because the design error would start to dominate.

## Conclusion

A fully connected neural network with ReLU activations, batch normalization, and dropout achieves 97.75% accuracy on the mfeat-factors 10-class digit classification task. The error estimation experiments confirm that resubstitution is severely optimistically biased (0.00% vs. true 1.75%), hold-out partially corrects this (0.62%) but remains optimistic due to single-split limitations, and 5-fold CV provides the most reliable estimate (2.50%) with slight pessimistic bias from the reduced training set size, as predicted by  $E[\hat{\varepsilon}_n] = E[\varepsilon_{n-n/k}]$  (Lecture 7). The quantitative RMS bound of  $\leq 2.80\%$  for hold-out with  $m = 320$  samples confirms that error estimation precision is fundamentally limited by validation set size. For model selection, cross-validation proved more discriminating than train-validation-test splitting: the selection probability theorem shows that TVT’s 240-sample validation set yields a vacuous bound, while CV effectively uses all 1600 samples. The selected model (Config C) achieves the best tradeoff in the complexity–sample size decomposition  $\varepsilon_{n,\mathcal{C}} = \varepsilon^* + \Delta_{\mathcal{C}} + \Delta_{n,\mathcal{C}}$  (Lecture 8), with higher capacity regularized by stronger dropout.